

Student Guide: Object-Oriented Sales Data Pipeline

Purpose

This guide helps students build a small end-to-end data engineering project using Python OOP, SQL, PostgreSQL, and Docker. The project is intentionally practical: students process sales data from raw CSV and JSON files, validate it, transform it, and load clean analytics tables into PostgreSQL.

By the end, students should understand how to organize a data pipeline into classes instead of writing one long script.

Learning Goals

1. Understand how OOP helps separate pipeline responsibilities.
2. Read raw data from CSV and JSON files.
3. Validate records and capture rejected records.
4. Transform raw records into analytics-ready data.
5. Load staging, dimension, and fact tables into PostgreSQL.
6. Run SQL queries for simple sales analytics.
7. Use Docker Compose to run PostgreSQL without installing it locally.

Project Scenario

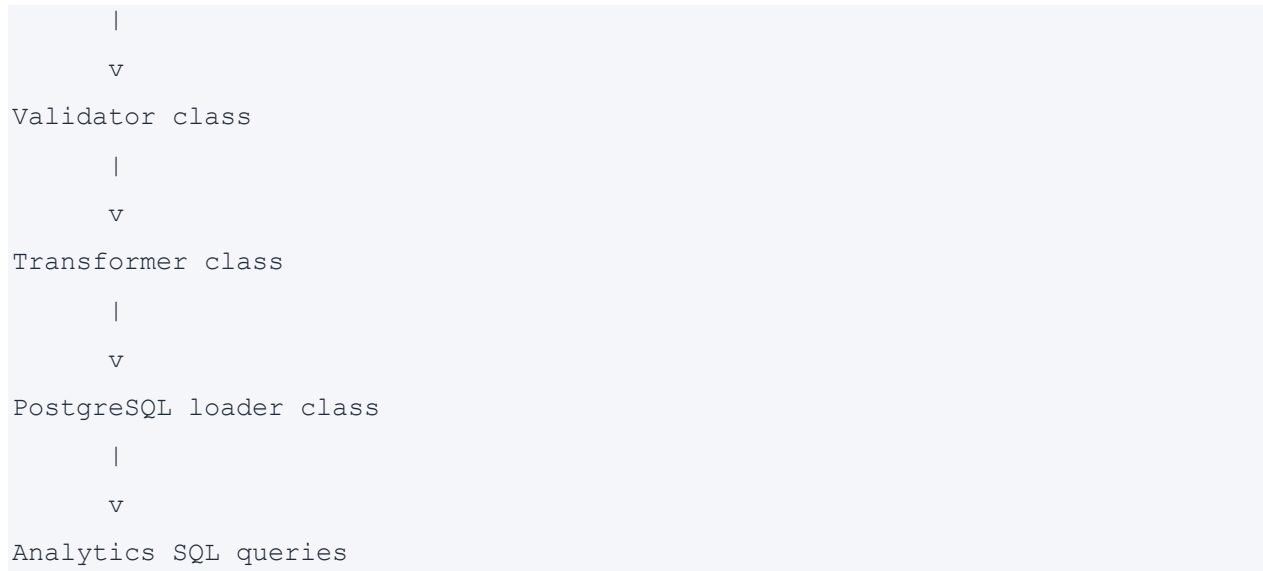
You work as a junior data engineer for a retail company. The business receives daily sales files and wants a repeatable process that can load clean data into a PostgreSQL database for analysis.

The pipeline should answer questions such as:

8. How much revenue did we make each day?
9. Which products generated the most revenue?
10. Which regions had the highest sales?
11. Which customer has the highest lifetime value?
12. Which raw records failed validation?

Final Pipeline Design

```
Raw sales CSV
Customer JSON
Product JSON
    |
    v
Extractor classes
```



OOP Class Responsibilities

Class	Main Responsibility
PipelineConfig	Reads file paths and database settings from environment variables.
CSVExtractor	Reads raw sales records from a CSV file.
JSONExtractor	Reads customer and product reference data from JSON files.
SalesDataValidator	Checks required fields, dates, numbers, prices, and foreign keys.
SalesDataTransformer	Converts valid records into customer, product, and sales fact models.
RejectedRecordWriter	Writes invalid records to a rejected-record CSV file.
DatabaseConnection	Opens, commits, rolls back, and closes PostgreSQL connections.
PostgresLoader	Creates database tables and inserts transformed records.
SalesPipeline	Orchestrates the full extract, validate, transform, and load workflow.

Step 1: Create the Project Structure

Create folders for source code, raw data, rejected data, and SQL:

```
mkdir -p src data/raw data/rejected sql docs
```

The target structure should look like this:

```
.
├── data/
│   ├── raw/
│   └── rejected/
├── docs/
├── sql/
├── src/
├── docker-compose.yml
├── main.py
└── requirements.txt
```

Step 2: Add Demo Input Data

Create three input files:

13. `data/raw/sales\_2026\_01.csv`
14. `data/raw/customers.json`
15. `data/raw/products.json`

The sales CSV should include both good and bad records. Bad records are useful because the validator needs something real to catch.

Example invalid cases:

16. Unknown customer ID
17. Unknown product ID
18. Quantity equal to zero
19. Negative unit price
20. Invalid date format
21. **Please Come up with more edge cases**

Checkpoint - *Verify important:

- **You should be able to check your output before load into the postgres**

- Or at least partial output
- Or have a test environment before the prod environment
- How to know if you answer is correct?
 - Came up with some solutions

Step 3(OPTIONAL): Define Data Models - Think of it as the project's data contract.

Create `src/models.py`.

Use dataclasses to represent the main data objects:

- 22. `Customer`
- 23. `Product`
- 24. `RawSalesRecord`
- 25. `ValidSalesRecord`
- 26. `FactSalesRecord`
- 27. `RejectedRecord`

Why this matters:

- 28. Dataclasses make records easier to read and pass between classes.
- 29. They give the project a clear data contract.
- 30. They help see the difference between raw, valid, transformed, and rejected data.

Step 4: Build Extractor Classes

Create `src/extractors.py`.

The extractor layer should contain:

- 31. An abstract `Extractor` base class.
- 32. A `CSVExtractor` class for sales CSV files.
- 33. A `JSONExtractor` class for customer and product JSON files.

OOP concept: ABSTRACTION ()

Step 5: Build the Validator

Create `src/validators.py`.

The validator should check:

- 34. Required fields are present.
- 35. `order\_id` and `quantity` are integers.

- 36. `order\_date` uses `YYYY-MM-DD`.
- 37. `unit\_price` and `discount\_rate` are decimal numbers.
- 38. `quantity` is greater than zero.
- 39. `unit\_price` is greater than zero.
- 40. `discount\_rate` is between zero and one.
- 41. `customer\_id` exists in the customer reference data.
- 42. `product\_id` exists in the product reference data.

43. More edge case we should validate

Valid records should become `ValidSalesRecord` objects. Invalid records should become `RejectedRecord` objects with a clear reason.

OOP concept:

Validation belongs in its own class because validation rules change over time. Keeping them separate prevents the pipeline orchestrator from becoming too large.

Step 6: Build the Transformer

Create `src/transformers.py`.

The transformer should:

- 44. Convert customer JSON rows into `Customer` objects.
- 45. Convert product JSON rows into `Product` objects.
- 46. Convert valid sales records into `FactSalesRecord` objects.
- 47. Calculate `gross\_sales`.
- 48. Calculate `discount\_amount`.
- 49. Calculate `net\_sales`.

Formula:

```
gross_sales = quantity * unit_price
discount_amount = gross_sales * discount_rate
net_sales = gross_sales - discount_amount
```

Step 7: Build the Loader

Create `src/loaders.py`.

The loader layer should contain:

- 50. ``RejectedRecordWriter`` for local rejected-record CSV output.
- 51. ``PostgresLoader`` for database inserts.

The PostgreSQL loader should create and load:

- 52. ``stg_sales``
- 53. ``dim_customers``
- 54. ``dim_products``
- 55. ``fact_sales``
- 56. ``etl_rejected_sales``
- 57. **If you have your own thought- please explain why(we can have snowflake schema, star schema, normalized table. As long as it make sense, you can do it)**

OOP concept:

Loading is separate from transformation. A transformer should not know whether the final destination is PostgreSQL, a CSV file, or a cloud warehouse.

Step 8: Add PostgreSQL Schema

Create ``sql/schema.sql``.

The schema should include ---- **if you designed your own stage use the one you designed:**

- 58. A staging table for valid sales records.
- 59. Dimension tables for customers and products.
- 60. A fact table for sales.
- 61. A rejected-record audit table.

Database design concept:

The project introduces a simple star schema:

- 62. Dimension tables describe business entities.
- 63. Fact tables store measurable business events.

Step 9: Add Docker Compose for PostgreSQL/ or Build in your local machine

Create ``docker-compose.yml``.

Use Docker Compose so students do not need to install PostgreSQL locally:

```
docker compose up -d
```

Connection settings:

```
Host: localhost
Port: 5432
Database: name_database
Username: postgres
Password: your_password
```

Stop the database:

```
docker compose down
```

Reset the database completely:

```
docker compose down -v
```

Step 10: Build the Pipeline Orchestrator(auto ingest)

Create `src/pipeline.py`.

The `SalesPipeline` class should coordinate the workflow:

64. Extract sales, customers, and products.
65. Transform customer and product reference data.
66. Validate raw sales records.
67. Transform valid sales records into fact records.
68. Write rejected records locally.
69. Load clean and rejected data into PostgreSQL, unless running in dry-run mode.
70. Return a summary of the run.

OOP concept:

The orchestrator coordinates classes, but it should not contain all the business logic itself.

Step 12: Run the Full Project

Expected tables in DBeaver – if you have your own table:

71. `stg\_sales`
72. `dim\_customers`
73. `dim\_products`
74. `fact\_sales`
75. `etl\_rejected\_sales`

Step 13: Run Analytics Queries

should verify: if the data is correct